
UniPool: A Globally Shared Expert Pool for Mixture-of-Experts

Minbin Huang¹ Han Shi² Chuanyang Zheng¹ Yimeng Wu²
 Guoxuan Chen³ Xingtong Yu¹ Yichun Yin² Hong Cheng¹

¹The Chinese University of Hong Kong

²Huawei Technologies

³The University of Hong Kong

Abstract

Modern Mixture-of-Experts (MoE) [22, 43] architectures allocate expert capacity through a rigid per-layer rule: each transformer layer owns a separate expert set. This convention couples depth scaling with linear expert-parameter growth and assumes that every layer needs isolated expert capacity. However, recent analyses and our routing probe challenge this allocation rule: replacing a deeper layer’s learned top- k router with uniform random routing drops downstream accuracy by only 1.0–1.6 points across multiple production MoE models. Motivated by this redundancy, we propose UNIPool, an MoE architecture that treats expert capacity as a global architectural budget by replacing per-layer expert ownership with a single shared pool accessed by independent per-layer routers. To enable stable and balanced training under sharing, we introduce a pool-level auxiliary loss that balances expert utilization across the entire pool, and adopt NormRouter to provide sparse and scale-stable routing into the shared expert pool. Across five LLaMA-architecture model scales (182M, 469M, 650M, 830M, and 978M parameters) trained on 30B tokens from the Pile, UNIPool consistently improves validation loss and perplexity over the matched vanilla MoE baselines. Across these scales, UNIPool reduces validation loss by up to 0.0386 relative to vanilla MoE. Beyond raw loss improvement, our results identify pool size as an explicit depth-scaling hyperparameter: reduced-pool UNIPool variants using only 41.6%–66.7% of the vanilla expert-parameter budget match or outperform layer-wise MoE at the tested scales. This shows that, under a shared-pool design, expert parameters need not grow linearly with depth; they can grow sublinearly while remaining more efficient and effective than vanilla MoE. Further analysis shows that UNIPool’s benefits compose with finer-grained expert decomposition. The code is open-sourced at <https://github.com/Centaurus-Alpha/UniPool>.

1 Introduction

Mixture-of-Experts (MoE) models have become a mainstream technique for scaling large language models (LLMs), enabling substantial parameter growth while maintaining nearly constant per-token computation [22, 43, 28, 11]. Conventional MoE design follows a rigid expert-budget allocation rule: each transformer layer owns its own set of expert FFNs, and a layer-specific router selects a sparse subset of those private experts for each token. This design, widely adopted in state-of-the-art MoE systems [23, 8, 9, 7], hard-codes a linear relationship between transformer depth and total expert parameters: adding layers necessarily allocates new private expert capacity.

Despite its widespread adoption, this allocation rule can be wasteful: *experts at different layers cannot be shared or reused*, even when they learn similar transformations. Section 3 synthesizes

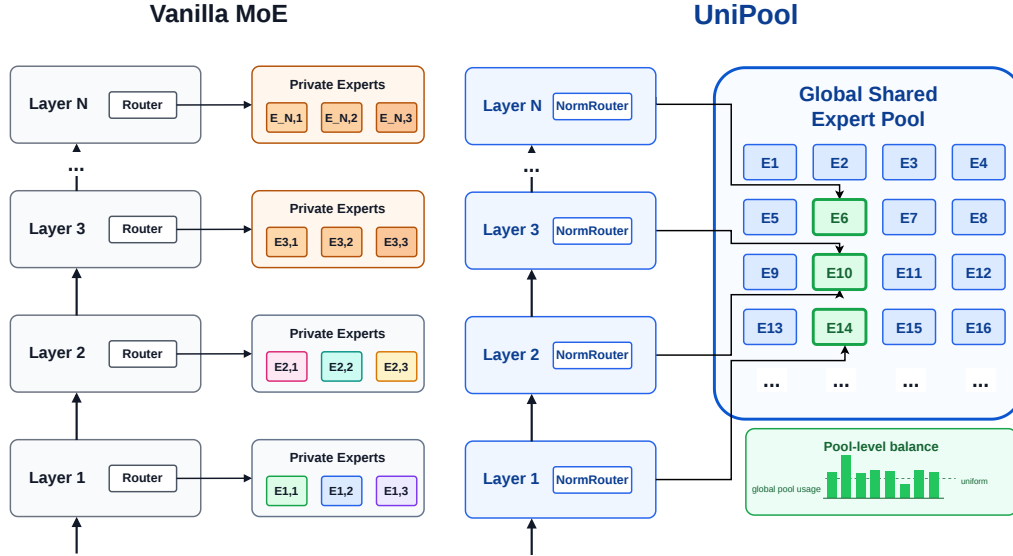


Figure 1: **UNIPool overview**. Vanilla MoE allocates a private expert set to each transformer layer, tying expert parameters to depth and preventing cross-layer reuse. UNIPool replaces layer-private ownership with a single global expert pool while keeping independent per-layer routers. Pool-level balancing aggregates utilization over the shared pool, preventing globally unused experts without forcing every layer to use every expert.

recent analyses of within-layer expert redundancy with our own routing-randomization probe on three production MoE models, where replacing the learned router in a single deep-half MoE layer with uniform random assignment drops downstream accuracy by only 1.0–1.6 points. These observations suggest that standard MoE training may duplicate expert functions across layer-private budgets rather than allocating expert capacity where it is most useful. This raises a fundamental question: *can expert capacity be treated as a global architectural budget shared across depth, while preserving layer-specific routing?* In this work, we propose **UNIPool (Unified Expert Pool)**, a MoE architecture with a globally shared expert pool, as illustrated in Fig. 1. This is non-trivial due to two key challenges.

First, **what is the right load-balancing objective when expert ownership becomes global?** In standard MoE [23, 7], auxiliary losses are applied independently at each layer to avoid *dead experts*: if a layer-private expert receives no tokens, its parameters are wasted. Under a shared pool, this layer-local notion of deadness is no longer aligned with where parameters are actually allocated. An expert unused by one layer may be frequently selected by other layers, so forcing every layer to use every shared expert conflicts with the goal of cross-layer reuse and layer-specific routing. We introduce a **pool-level auxiliary loss** that balances utilization at the granularity where parameters are actually owned: the global expert pool. Instead of computing utilization statistics independently for each layer, we aggregate token-to-expert assignments across layers and apply a single objective over the shared pool. This design prevents globally dead experts while allowing different layers to specialize on different subsets of experts.

Second, **how to maintain stable and effective routing into a global expert budget?** Conventional softmax-based routers are designed for layer-specific experts. In UNIPool, routers at different depths all select from the same larger expert pool, so layer-dependent logit scales can translate into inconsistent routing sharpness and unstable competition among shared experts. We therefore adopt **NormRouter** [62], which replaces softmax gating with an L2-normalize-then-ReLU [34] scoring function combined with a learnable scaling factor. This formulation is well matched to shared-pool routing: normalization makes scores less sensitive to layer-specific hidden-state scale, ReLU induces

sparse competition over the large pool, and the learnable scale lets each router adjust routing strength during training.

In summary, our contributions are as follows:

- **Redundancy in layer-wise experts.** We identify per-layer expert ownership as a rigid MoE allocation rule that ties expert parameters linearly to depth, and show through a routing-randomization probe that deeper layer-private experts can be substantially redundant.
- **A global expert pool.** We propose UNIPool, which replaces layer-private expert sets with a single shared expert pool accessed by independent per-layer routers, enabling cross-layer expert reuse while preserving layer-specific routing.
- **Pool-level balancing and routing.** We introduce a pool-level auxiliary loss and adopt NormRouter as a co-design for shared-pool MoE, balancing utilization over the shared pool while providing sparse, scale-stable routing that is well suited to a larger expert pool.
- **Sublinear expert scaling.** Across five model scales trained on 30B tokens, UNIPool consistently improves over vanilla MoE; reduced-pool variants using only 41.6%–66.7% of the vanilla expert-parameter budget match or outperform layer-wise MoE.

2 Related Work

Sparse MoE and scaling. The modern MoE paradigm for language models was established by sparsely gated expert layers [43], then scaled through top-1 routing in Switch Transformer [11], expert-parallel distributed training in GShard [28], and stability improvements such as ST-MoE’s router z-loss [66]. Recent large-scale systems including Mixtral [23] and the DeepSeek series [7, 8, 9] further show that sparse expert capacity is an effective way to scale language models. Complementary work studies expert granularity and scaling laws, finding that a larger number of smaller experts can improve performance when paired with appropriate routing [25], with extreme variants considering up to a million experts [17]. These works largely retain per-layer expert ownership; UNIPool instead studies whether expert capacity can be reused across depth through a global shared pool.

Routing and load balancing. Effective MoE training depends on routing mechanisms that select useful experts while keeping utilization balanced. The standard approach uses softmax routing with the Switch auxiliary loss, which penalizes correlation between per-expert token fractions and routing probabilities within each layer [11]. Other routing designs enforce or encourage balance through expert choice [64], linear assignment in BASE layers [29], deterministic hash routing [40], sigmoid gating [9], or ReLU-based sparse routing [32]. UNIPool addresses a different balancing regime: once experts are shared across layers, dead-expert prevention should be defined over the global pool rather than within every layer, so we combine a pool-level auxiliary loss with NormRouter’s L2-normalized ReLU scores.

Parameter sharing and expert reuse. Cross-layer parameter sharing has been explored as a way to improve parameter efficiency in Transformers, including Universal Transformers [10] and ALBERT [27]. Those models share broad parameters across depth, whereas UNIPool applies sharing selectively to MoE expert FFNs while retaining layer-specific attention blocks and routers. A closer line of work, MoEUT [6], cyclically repeats a small group of shared transformer blocks across depth with per-layer entropy balancing; UNIPool instead shares only the FFN experts as a single global pool, leaves routers and attention per-layer, and balances utilization at the pool level. This targeted sharing matches the structure of sparse MoE models: expert FFNs constitute a large fraction of stored parameters, but routers at different depths can still learn distinct token-to-expert policies.

3 Motivating Observation: Expert Redundancy in Deep MoE Layers

Recent analyses of trained MoEs document substantial within-layer expert redundancy from multiple angles: same-layer expert weight matrices in Qwen and DeepSeek MoEs share a dominant subspace with pairwise cosine similarity above 0.9 [20], tokens re-routed to the most-similar same-layer expert preserve accuracy with up to $2\times$ decoding speedup on Qwen1.5-MoE, DeepSeek-V2-Lite, Qwen3-30B-A3B, and OLMoE [54], and pruning roughly half the experts in Mixtral $8\times 7B$ costs

Table 1: Routing redundancy under single-layer randomization in production MoE models. Accuracy (%) is reported on five downstream benchmarks. TOP-K: original learned top- k routing; RANDOM: mean accuracy after randomizing one deep-half MoE layer at a time and averaging across layers. Avg is the unweighted mean, with drops measured relative to TOP-K.

Model	Routing	ARC-E	ARC-C	PIQA	HellaSwag	WinoGrande	Avg
<i>Production MoE models</i>							
Qwen1.5-MoE	TOP-K	69.23	44.20	80.47	77.30	68.43	67.92
	RANDOM	66.76	42.19	79.07	76.08	67.34	66.29 (−1.6)
DeepSeek-V2-Lite	TOP-K	58.59	33.02	67.57	56.82	54.93	54.19
	RANDOM	57.23	32.08	65.88	55.41	54.57	53.03 (−1.2)
Qwen3-30B-A3B	TOP-K	79.50	55.97	80.79	77.70	71.11	73.02
	RANDOM	78.67	54.98	79.71	76.85	70.10	72.06 (−1.0)

only $\sim 8\%$ relative quality, with the strongest intra-layer similarity concentrated in deep layers [1]. These works characterize redundancy in expert *parameters* and *outputs*, but treat it as a target for post-hoc compression while keeping per-layer expert ownership intact. We complement this picture by probing the *router* itself: if a deep layer’s experts carry distinct specializations, randomizing the routing decision should noticeably hurt accuracy. On three production MoEs (Qwen1.5-MoE, DeepSeek-V2-Lite, Qwen3-30B-A3B) we replace the learned top- k router in a single deep-half MoE layer with uniform random assignment, sweep the intervention over every deep-half layer, and report the average downstream accuracy in Table 1, where TOP-K denotes the original learned router and RANDOM the single-layer deep-half randomization.

The drop is only 1.0–1.6 points across all three models: the choice among same-layer experts carries limited local information at depth, indicating that the per-layer router is not committing to a sharp functional partition over its private expert set. This routing observation aligns with the parameter- and output-level evidence above: same-layer expert parameters and outputs are highly similar [20, 54, 1] with the strongest similarity in deep layers [1], and the router that selects among them adds little task-level signal at those depths (Table 1). Together, these signals suggest that strict per-layer ownership encourages every block to independently rediscover similar transformations from a thin gradient signal, producing the deep-layer redundancy that pruning and similar-expert re-routing methods then remove post hoc—addressing the symptom rather than the cause. The structural alternative is to drop the ownership constraint entirely and route every layer into a single shared pool of experts: each expert then accumulates gradients from L layers rather than one, depth-induced redundancy is converted into architectural reuse instead of being trimmed away after training, and the total expert-parameter count decouples from depth. We return to this question empirically in Section 6.1, where the same routing-randomization probe applied to our own UNIPOL models shows a substantially larger drop than on vanilla MoE—consistent with the view that sharing actively breaks the redundancy that single-layer randomization fails to disrupt; Appendix Table 11 reports per-task results.

4 Method

We describe the three components of UNIPOL: the shared expert pool architecture (Section 4.1), the pool-level auxiliary loss (Section 4.2), and our use of NormRouter for shared-pool routing (Section 4.3).

4.1 Global Shared Expert Pool

In a standard MoE transformer with L layers and E experts per layer, each layer l maintains its own set of expert FFNs $\{e_{l,1}, \dots, e_{l,E}\}$ and a router r_l . The FFN output at layer l for token x is:

$$\text{FFN}_l(x) = \sum_{i \in \text{Top-}k(r_l(x))} g_{l,i}(x) \cdot e_{l,i}(x), \quad (1)$$

where $g_{l,i}(x)$ is the gating weight assigned by router r_l to expert i for token x .

In UNIPOL, we replace the L separate expert sets with a single *global shared pool* $\mathcal{E} = \{e_1, \dots, e_M\}$ of M expert FFNs. Each layer retains its own router r_l , which routes tokens into this shared pool:

$$\text{FFN}_l(x) = \sum_{i \in \text{Top-}k(r_l(x))} g_{l,i}(x) \cdot e_i(x). \quad (2)$$

The key difference from Eq. (1) is that *expert parameters are shared*: e_i in Eq. (2) is the same module regardless of which layer l invokes it. Routers r_l remain per-layer because different depths in the residual stream require different routing patterns, even though the underlying expert computations are shared. The pool size M is a configuration choice; in the main experiments it is set to match the vanilla MoE expert-parameter budget while preserving dense-equivalent active FFN compute (Section 5.1).

4.2 Pool-Level Auxiliary Loss

Mismatch of per-layer auxiliary loss under sharing. The standard Switch Transformer auxiliary loss [11] for a single layer l is:

$$\mathcal{L}_{\text{aux}}^{(l)} = \alpha \cdot E \cdot \sum_{i=1}^E f_i^{(l)} \cdot P_i^{(l)}, \quad (3)$$

where $f_i^{(l)}$ is the fraction of tokens dispatched to expert i and $P_i^{(l)}$ is the mean routing probability for expert i , both within layer l . In layer-private MoE, this layer-local objective matches the parameter ownership structure: a dead expert within layer l means that layer’s private expert parameters are unused. Under a shared pool, however, expert parameters are owned globally rather than by a single layer. An expert that is unused by layer l may be frequently used by other layers, so treating it as dead within layer l violates the original purpose of load balancing and unnecessarily forces every layer to spread traffic over the entire pool. The appropriate dead-expert criterion is therefore global pool utilization, not per-layer utilization.

Pool auxiliary loss. For a shared pool of M experts, we define the *global average* token fraction across all L sharing layers:

$$\bar{f}_i = \frac{1}{L} \sum_{l=1}^L f_i^{(l)}, \quad (4)$$

and the pool-level loss as:

$$\mathcal{L}_{\text{pool}} = \alpha_{\text{pool}} \cdot M \cdot \sum_{i=1}^M \bar{f}_i \cdot \bar{P}_i, \quad (5)$$

where $\bar{P}_i = \frac{1}{L} \sum_l P_i^{(l)}$ is the global average routing probability. Because $\bar{f}_i$ is the same for all layers, the pool loss decomposes into per-layer contributions that can be computed independently:

$$\mathcal{L}_{\text{pool}} = \frac{1}{L} \sum_{l=1}^L \alpha_{\text{pool}} \cdot M \cdot \sum_{i=1}^M \bar{f}_i \cdot P_i^{(l)}. \quad (6)$$

In practice, we compute the global token-distribution statistic one micro-batch behind to avoid cross-layer tensor dependencies while retaining the decomposed objective; Appendix G gives the implementation details.

4.3 NormRouter

Standard MoE routers compute gating weights via softmax over logits $z = Wh$, where $W \in \mathbb{R}^{E \times d}$ and $h \in \mathbb{R}^d$ is the token hidden state. We adopt NormRouter (KERN) [62] in place of softmax routing, computing scores as:

$$s_i = \sigma \cdot c \cdot \max\left(0, \frac{z_i}{\|z\|_2 + \epsilon}\right), \quad (7)$$

where σ is a learnable scalar (initialized to 1), c is a fixed constant determined by Monte Carlo estimation (Appendix H), and ϵ is a small constant for numerical stability.

Score function properties. The L2 normalization ensures that score magnitudes are bounded regardless of the input scale. This is particularly useful in UNIPOL because routers at different depths all select from the same large expert pool, while their hidden-state norms and logit scales can differ substantially. Softmax routing can make such scale differences translate into inconsistent routing sharpness across layers; NormRouter instead makes routing depend primarily on the logit direction, with the learnable scale σ absorbing the desired magnitude. The ReLU activation produces naturally sparse scores—roughly half of the experts receive zero score for any given token—which sharpens the routing distribution without requiring explicit sparsification. The fixed constant c calibrates the initial top- k score scale so that selected routing scores have approximately unit magnitude; Appendix H gives the expectation and sampling procedure.

Top- k selection and auxiliary losses. After computing scores via Eq. (7), top- k experts are selected based on the highest scores. The NormRouter is fully compatible with both the standard per-layer auxiliary loss and our pool-level auxiliary loss, which operate on the routing scores s_i in place of the softmax probabilities.

5 Experiments

5.1 Experimental Setup

Scale	Arch.	Method	Loss ↓	PPL ↓
182M	12/768	Dense	2.042	7.708
		Vanilla MoE	1.9317	6.9012
		UNIPOL	1.9029	6.7058
469M	24/1024	Dense	1.886	6.593
		Vanilla MoE	1.7982	6.0388
		UNIPOL	1.7636	5.8334
650M	36/1024	Dense	1.8318	6.2453
		Vanilla MoE	1.7568	5.7940
		UNIPOL	1.7260	5.6186
830M	48/1024	Dense	1.8032	6.0694
		Vanilla MoE	1.7309	5.6458
		UNIPOL	1.6923	5.4320
978M	24/1536	Dense	1.822	6.184
		Vanilla MoE	1.7171	5.5683
		UNIPOL	1.6999	5.4736

Table 2: Main results after 30B training tokens under the default 8E/top-1 MoE configuration.

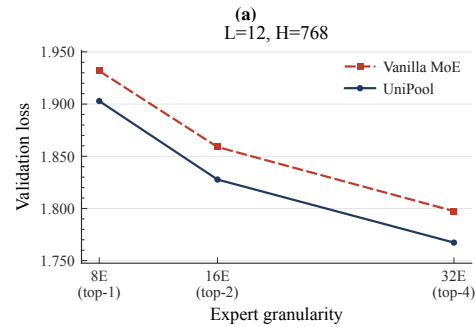
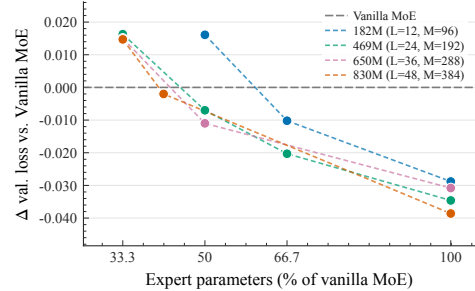


Figure 2: Efficiency and granularity sweeps for UniPool.

Model architecture. We use LLaMA-style transformer backbones [49] and evaluate five active-parameter scales from 182M to 978M. Full architectural details, including layer counts, hidden sizes, attention heads, and FFN dimensions, are provided in Table 6 (Appendix B).

MoE configurations and parameter matching. The vanilla MoE baseline uses 8 private expert FFNs per layer with top-1 softmax routing. UNIPOL replaces these private layer-wise experts with a single global pool of $M = 8L$ shared experts while preserving top-1 active expert computation per layer. Thus vanilla MoE and UNIPOL are matched in total expert FFNs and per-token expert FLOPs; the comparison isolates expert ownership, routing, and balancing rather than changing active compute. Unless otherwise stated, vanilla MoE uses the standard per-layer auxiliary loss, while UNIPOL uses the pool-level auxiliary loss and NormRouter. Table 7 (Appendix B) gives the full configuration comparison.

Table 3: Zero-shot downstream evaluation (accuracy %). The top block reports default 8E/top-1 results across model scales; the bottom block reports expert-granularity sweeps on the 182M backbone. **Bold** marks the better method; “Avg” is the unweighted mean.

Setting	Scale	Method	ARC-E $\uparrow$	ARC-C $\uparrow$	PIQA $\uparrow$	HellaSwag $\uparrow$	WinoGrande $\uparrow$	LAMBADA $\uparrow$	RACE $\uparrow$	Avg $\uparrow$
Main scales (default 8E / top-1 MoE)										
8E / top-1	182M	Vanilla MoE	45.71	19.97	63.11	29.98	50.99	32.78	28.61	38.74
		UniPOOL	46.72	20.48	64.36	30.66	50.99	34.56	29.47	39.61
8E / top-1	469M	Vanilla MoE	50.51	21.08	66.32	32.72	51.14	40.21	29.38	41.62
		UniPOOL	53.16	21.42	67.30	33.90	52.72	42.19	31.10	43.11
8E / top-1	650M	Vanilla MoE	51.94	21.25	67.03	34.53	53.04	43.74	29.76	43.04
		UniPOOL	52.02	22.61	67.90	35.55	52.49	44.28	31.67	43.79
8E / top-1	830M	Vanilla MoE	52.53	23.89	68.93	35.36	52.33	43.14	30.53	43.82
		UniPOOL	56.57	25.00	68.77	36.90	52.49	47.37	32.63	45.67
8E / top-1	978M	Vanilla MoE	53.24	23.21	68.01	35.83	52.01	44.63	30.43	43.91
		UniPOOL	54.34	22.27	69.21	36.19	52.17	44.94	29.38	44.07
Expert-granularity sweep at 182M										
16E / top-2	182M	Vanilla MoE	48.82	21.59	65.07	31.83	49.72	36.48	28.80	40.33
		UniPOOL	49.24	20.22	65.45	32.33	54.22	37.86	29.19	41.22
32E / top-4	182M	Vanilla MoE	50.08	21.08	66.43	32.91	51.54	39.41	29.00	41.49
		UniPOOL	52.44	22.27	67.41	34.32	50.51	40.77	30.62	42.62

Implementation and Training details We implement UNIPool in Megatron-LM [45] by instantiating the expert pool once and reusing the same experts module across MoE layers, while keeping routers layer-specific. All models are trained on the Pile dataset [15] for 60,000 iterations with batch size 512 and sequence length 1,024, totaling approximately 30B tokens. We use AdamW [31] with a cosine learning-rate schedule and bf16 Megatron-LM training [45]; Appendix D reports the complete optimizer and systems settings. For variance checks, the 182M main results are averaged over three random seeds, while larger-scale results use one run per configuration due to training cost.

Expert-size scaling experiment. To test whether UNIPool composes with finer expert granularity, we run an additional granularity sweep based on 182M model over 16E/top-2 and 32E/top-4 MoE configurations. These settings change total and active expert parameters, so they are analyzed separately from the matched main comparisons.

5.2 Main Results: UNIPool vs. Vanilla MoE

Table 2 reports the validation loss and perplexity for the dense baseline, vanilla MoE, and UNIPool at five model scales. UNIPool consistently outperforms both baselines across all scales.

Consistent improvement across scales. The improvement from UNIPool over vanilla MoE is consistent at all five scales, with validation loss reductions of 0.0288 (182M), 0.0346 (469M), 0.0308 (650M), 0.0386 (830M), and 0.0172 (978M). Both MoE methods substantially outperform the dense baseline (e.g., 1.9029 vs. 2.042 at 182M), confirming that sparse expert routing is effective, and UNIPool further widens this gap by making better use of the shared expert capacity. The 830M/978M pair is especially informative because it changes the architecture shape rather than only the nominal scale. The 978M model allocates capacity primarily to width (24 layers, hidden size 1536), whereas the 830M model uses a deeper stack (48 layers, hidden size 1024) with fewer active parameters and fewer stored UNIPool parameters.¹ UNIPool achieves both its largest loss reduction over vanilla MoE in the deeper 830M model (-0.0386) and a lower absolute validation loss than the wider 978M UNIPool model (1.6923 vs. 1.6999), despite the latter having a larger active and stored parameter budget. This supports a budget-allocation view of shared-pool MoE: for this architecture family, allocating capacity toward depth and reusable expert pools can be more effective than allocating it primarily to width, because additional layers create more sites that can reuse the global expert pool. Under this view, the smaller 978M gap is expected rather than contradictory; it suggests that UNIPool’s marginal gain is strongest when the architecture exposes more cross-layer expert-reuse opportunities, not merely when the total parameter count increases.

¹Appendix Table 6 reports the stored UNIPool parameter counts: 5.081B/5.742B for the 830M/978M configurations.

Total-parameter efficiency: matching the baseline with a smaller pool. Figure 2(a) plots validation-loss change against the fraction of vanilla expert parameters retained in the shared pool. The key pattern is that UNIPool can beat the layer-private baseline before reaching the matched expert budget: the smallest winning pools use 66.7% of vanilla expert parameters at 182M, 50% at 469M and 650M, and 41.6% at 830M. Thus, under the same top-1 active expert compute, pool size becomes a practical depth-scaling knob rather than forcing expert parameters to grow linearly with the number of layers.

We further test whether the shared pool can be shrunk below the matched vanilla budget by training reduced-pool UNIPool variants at 182M ($M=64, 48$; 66.7%/50% of the matched expert parameters), 469M ($M=128, 96, 64$; 66.7%/50%/33.3%), 650M ($M=144, 128, 96$; 50%/44.4%/33.3%), and 830M ($M=160, 128$; 41.6%/33.3%), keeping top-1 routing so active parameters stay matched. Figure 2(a) reports validation-loss change relative to each scale’s vanilla MoE baseline. **At every tested scale, a sub-vanilla pool surpasses the layer-private baseline:** 66.7% at 182M (1.9215 vs. 1.9317), 50% at 469M (-0.007) and 650M (-0.011), and 41.6% at 830M (-0.013); the smallest winning fraction shrinks monotonically with depth, so deeper backbones tolerate progressively smaller shared pools. This directly tests the budget-allocation view motivated in Section 3: if vanilla MoE’s layer-private expert sets duplicate useful functions, then a smaller globally shared pool should be able to match or surpass the larger per-layer allocation. The reduced-pool results support this prediction, suggesting that the vanilla organization is over-provisioned at the tested scales and that sharing can turn redundant private capacity into reusable global capacity. These reduced-pool results turn pool size into an *explicit scaling hyperparameter*: at the tested scales, expert parameters can grow sublinearly with the number of layers while preserving or improving quality, freeing budget that can be reinvested into a deeper backbone or a larger pool.

Granularity scaling. Figure 2(b) further shows that the gain composes with finer-grained MoE: at the 182M scale, UNIPool outperforms the matched vanilla MoE baseline under all three configurations (8E/top-1, 16E/top-2, 32E/top-4), and both methods improve with larger expert counts, consistent with prior scaling results for fine-grained MoE [25].

Training dynamics. The endpoint gains are also visible throughout optimization: after warmup, UNIPool remains below vanilla MoE at the 182M, 469M, and 650M scales, and the sharing-scope sweep follows the same ordering as the final validation losses. Because these curves support rather than define the main result, we place them in Appendix C; Appendix Figure 4 gives the scale-wise trajectories and Appendix Figure 4d shows the sharing-scope trajectory.

5.3 Downstream Evaluation

To verify that the perplexity improvements translate to task-level gains, we evaluate all models on seven standard zero-shot benchmarks: ARC-Easy and ARC-Challenge [5], PIQA [2], HellaSwag [57], WinoGrande [41], LAMBADA [36], and RACE [26]. Table 3 reports raw accuracy (acc) for each task.

5.4 Ablation Studies

To understand the contribution of each component, we conduct ablation studies at the 182M scale. For the sharing-scope variants, G denotes the number of expert-pool groups across depth: $G=12$ recovers layer-private vanilla MoE at 12 layers, while $G=1$ is the fully shared UNIPool pool.

Table 5 summarizes the component ablations and sharing-scope variants. The main takeaway is that sharing requires a matched routing and balancing design: a shared pool with the original per-layer auxiliary loss underperforms vanilla MoE (1.9480 vs. 1.9317), while replacing it with the pool-level auxiliary loss improves the loss to 1.9180. Replacing the vanilla softmax router with NormRouter alone slightly worsens validation loss (1.9375 vs. 1.9317), indicating that the gains of UNIPool are not explained by a stronger router in the layer-private MoE setting. We hypothesize that NormRouter is more useful when routing over a larger and effectively sparser candidate set, as in the shared-pool setting where all layers compete for the same global expert pool. The aux-free vanilla baseline reaches 1.9239, so simply loosening load balancing is not enough to match the full shared-pool design. Combining the shared pool, pool-level auxiliary loss, and NormRouter gives the best result in the table (1.9029). The sharing-scope rows further show that intermediate grouping already improves

Model	Routing	Avg
Vanilla MoE (469M)	TOP-K	45.10
	RANDOM	43.83 (−1.3)
UNIPOOL (469M)	TOP-K	47.16
	RANDOM [†]	43.10 (−4.1)
Vanilla MoE (978M)	TOP-K	48.13
	RANDOM	46.64 (−1.5)
UNIPOOL (978M)	TOP-K	48.35
	RANDOM [†]	44.25 (−4.1)

Table 4: Avg-only routing-randomization results on our trained models. [†] denotes the matched top-8 random protocol for UNIPOOL. Full per-task values are in Appendix Table 11.

Configuration	Loss	Δ
<i>Components and sharing endpoints</i>		
Vanilla MoE + softmax ($G=12$)	1.9317	-
Vanilla MoE + NormRouter	1.9375	+0.0058
V-MoE, sigmoid, aux-free	1.9239	-0.0078
Shared + layer aux + softmax	1.9480	+0.0163
Shared + pool aux + softmax	1.9180	-0.0137
UNIPOOL ($G=1$)	1.9029	-0.0288
<i>Intermediate sharing scope</i>		
$G=6$	1.9121	-0.0196
$G=4$	1.9099	-0.0218
$G=2$	1.9213	-0.0104

Table 5: Ablation study at 182M; Δ is relative to vanilla MoE. Endpoint rows also correspond to the $G=12$ and $G=1$ sharing-scope settings.

over vanilla MoE, with global sharing ($G=1$) performing best; the corresponding training trajectories are shown in Appendix Figure 4d.

6 Analysis

Beyond the main results, we provide three analytical lenses on UNIPOOL’s behavior: a routing-randomization comparison with vanilla MoE (Section 6.1), an expert-reuse and budget-allocation view of cross-layer sharing (Section 6.2), and an empirical study of expert utilization and routing diversity under the shared pool (Section 6.3).

6.1 Routing Sensitivity in Vanilla MoE vs. UNIPOOL

Table 4 tests whether routing decisions become more load-bearing after expert sharing. In vanilla MoE, randomizing one deep-half layer reduces average accuracy by only 1.3/1.5 points at 469M/978M, matching the production-model redundancy pattern from Section 3. For UNIPOOL, the cardinality-matched top-8 randomization drops average accuracy by 4.1 points at both scales. This supports the central claim that the shared pool reduces expert substitutability: UNIPOOL routers select reusable computations that are less interchangeable than layer-private deep experts. Full per-task values and full-pool randomization variants are reported in Appendix Table 11.

The two routing-randomization results—the small drop on vanilla MoE in Section 3 and the much larger drop on UNIPOOL below—are two sides of the same redundancy story rather than a contradiction. In a layer-private MoE, every layer trains its own expert bank from a thin per-block gradient signal, so deep-layer experts converge to similar transformations [20, 54, 1] and effectively lose specialization: any one of them is roughly substitutable for any other, so randomly picking among them costs little (−1.3/−1.5 on our own vanilla models). UNIPOOL removes this slack by exposing every expert to gradient signal from L layers and forcing all layers to compete over a single global pool; experts that survive this competition specialize, and the per-layer router’s choice becomes load-bearing.

Concretely, Table 4 repeats the routing-randomization intervention on our own 469M and 978M models, which are trained under matched data and optimizer settings. Vanilla MoE again loses only 1.3/1.5 average accuracy points when one deep-half layer is randomized, matching the production-model pattern from Section 3. For UNIPOOL, we use a cardinality-matched intervention that samples from each layer’s top-8 most-used shared experts; the drop rises to 4.1 points at both scales. Under this matched protocol, the per-layer router in UNIPOOL carries substantially more information about which reusable computation to invoke at each depth, providing structural evidence that the shared pool has converted depth-induced redundancy into specialization. Appendix Table 11 also reports the standard full-pool random protocol, which samples uniformly from all shared experts and complements the cardinality-matched comparison with an unrestricted pool-wide intervention.

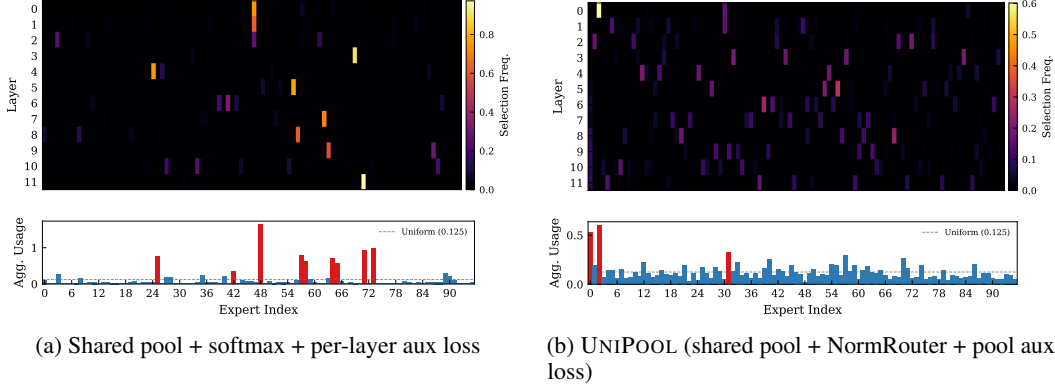


Figure 3: Expert utilization at the 182M scale: per-layer auxiliary loss leads to global expert collapse, while UNIPool restores balanced shared-pool usage.

6.2 Expert Reuse and Budget Allocation

The sharing-scope and reduced-pool results suggest that UNIPool’s gains are tied to cross-layer reuse rather than simply adding a stronger router. Viewed as routed compositions, top-1 MoE selects a length- L sequence of expert transformations for each token. UNIPool relaxes the vanilla constraint that the l -th choice must come from layer l ’s private expert set, allowing the same expert functions to be reused across depths. Under matched top-1 compute, vanilla MoE touches one private expert tensor per layer, whereas UNIPool can route multiple layers to the same shared expert. For full-pool UNIPool models, the fraction of unique expert weights touched by a token falls from 94.1% at 12 layers to 89.5% at 24 layers and 82.7% at 36 layers, indicating increasing reuse with depth; Appendix E gives the full accounting. This also explains why pool size becomes a scaling hyperparameter: a smaller pool increases reuse and exposes each expert to gradients from more layers, while an overly small pool can introduce interference among depth-specific demands. The reduced-pool experiments in Figure 2(a) show that, at the tested scales, this tradeoff can favor sublinear expert-parameter growth with depth.

6.3 Expert Utilization and Routing Diversity

Expert utilization balance. Figure 3 illustrates why pool-level auxiliary loss is critical for the shared-pool architecture. Both configurations share the same global expert pool; they differ only in the auxiliary loss and router design. In each panel, the top heatmap shows per-layer expert selection frequency, while the bottom bar plot aggregates usage across all layers against the uniform reference line. With per-layer auxiliary loss and softmax routing (Figure 3a), aggregate traffic collapses onto a small subset of shared experts, showing that the layer-local balancing objective is misaligned with global parameter ownership. UNIPool with pool-level auxiliary loss and NormRouter (Figure 3b) restores balanced global usage while preserving layer-specific routing patterns in the heatmap. Together with the component ablation in Table 5, this analysis connects the stabilization components to the shared-pool design: the pool loss supplies the right utilization objective, while NormRouter provides the sparse, scale-stable scores used by each layer to access the shared pool.

7 Conclusion

We introduced UNIPool, a Mixture-of-Experts architecture that replaces layer-private expert ownership with a global shared pool trained using pool-level balancing and NormRouter. Across five model scales, UNIPool improves validation loss and perplexity over matched vanilla MoE baselines, while reduced-pool variants can outperform vanilla MoE with only 41.6%–66.7% of its expert-parameter budget. These results suggest that MoE expert capacity can be allocated as a reusable global budget whose pool size scales sublinearly with depth, rather than being tied rigidly to per-layer expert ownership.

References

- [1] Sikai Bai, Haoxi Li, Jie Zhang, Zicong Hong, and Song Guo. Diep: Adaptive mixture-of-experts compression through differentiable expert pruning. *arXiv preprint arXiv:2509.16105*, 2025.
- [2] Yonatan Bisk, Rowan Zellers, Jianfeng Gao, Yejin Choi, et al. PIQA: Reasoning about physical intuition by question answering. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pages 7432–7439, 2020.
- [3] Aidan Clark, Diego de las Casas, Aurelia Guy, Arthur Sherrington, Mia Saber, Jay Sherburn, Jean Sherrington, Michael Sherrington, et al. Unified scaling laws for routed language models. *arXiv preprint arXiv:2202.01169*, 2022.
- [4] Christopher Clark, Kenton Lee, Ming-Wei Chang, Tom Kwiatkowski, Michael Collins, and Kristina Toutanova. BoolQ: Exploring the surprising difficulty of natural yes/no questions. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics*, pages 2924–2936, 2019.
- [5] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? try ARC, the AI2 reasoning challenge. *arXiv preprint arXiv:1803.05457*, 2018.
- [6] Róbert Csordás, Kazuki Irie, Jürgen Schmidhuber, Christopher Potts, and Christopher D. Manning. MoEUT: Mixture-of-experts universal transformers. In *Advances in Neural Information Processing Systems*, 2024.
- [7] Damai Dai, Chengqi Deng, Chenggang Zhao, R.X. Xu, Huazuo Gao, Deli Chen, Jiashi Li, Wenge Zeng, Xingkai Yu, Y. Wu, et al. DeepSeekMoE: Towards ultimate expert specialization in mixture-of-experts language models. *arXiv preprint arXiv:2401.06066*, 2024.
- [8] DeepSeek-AI. DeepSeek-V2: A strong, economical, and efficient mixture-of-experts language model. *arXiv preprint arXiv:2405.04434*, 2024.
- [9] DeepSeek-AI. DeepSeek-V3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.
- [10] Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Łukasz Kaiser. Universal transformers. In *International Conference on Learning Representations*, 2019.
- [11] William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022.
- [12] Kuniyiko Fukushima. Neocognitron: A self-organizing neural network model for a mechanism of pattern recognition unaffected by shift in position. *Biological cybernetics*, 36(4):193–202, 1980.
- [13] Kuniyiko Fukushima. Visual feature extraction by a multilayered network of analog threshold elements. *IEEE Transactions on Systems Science and Cybernetics*, 5(4):322–333, 2007.
- [14] Trevor Gale, Deepak Narayanan, Cliff Young, and Matei Zaharia. MegaBlocks: Efficient sparse training with mixture-of-experts. *Proceedings of Machine Learning and Systems*, 5, 2023.
- [15] Leo Gao, Stella Biderman, Sid Black, Laurence Golding, Travis Hoppe, Charles Foster, Jason Phang, Horace He, Anish Thite, Noa Nabeshima, et al. The Pile: An 800GB dataset of diverse text for language modeling. *arXiv preprint arXiv:2101.00027*, 2020.
- [16] Mor Geva, Roei Schuster, Jonathan Berant, and Omer Levy. Transformer feed-forward layers are key-value memories. *arXiv preprint arXiv:2012.14913*, 2020.
- [17] Xu Owen He. Mixture of a million experts. *arXiv preprint arXiv:2407.04153*, 2024.
- [18] Dan Hendrycks and Kevin Gimpel. Gaussian error linear units (gelus). *arXiv preprint arXiv:1606.08415*, 2016.

- [19] Alston S Householder. A theory of steady-state activity in nerve-fiber networks: I. definitions and preliminary lemmas. *The bulletin of mathematical biophysics*, 3(2):63–69, 1941.
- [20] Ruijun Huang, Fang Dong, Xin Zhang, Hengjie Cao, Zhendong Huang, Anrui Chen, Jixian Zhou, Mengyi Chen, Yifeng Yang, Mingzhi Dong, et al. Sd-moe: Spectral decomposition for effective expert specialization. *arXiv preprint arXiv:2602.12556*, 2026.
- [21] Changho Hwang, Wei Cui, Yifan Xiong, Ziyue Yang, Ze Liu, Han Hu, Zilong Wang, Rafael Salas, Jithin Jose, Parijat Ram, et al. Tutel: Adaptive mixture-of-experts at scale. *Proceedings of Machine Learning and Systems*, 5, 2023.
- [22] Robert A. Jacobs, Michael I. Jordan, Steven J. Nowlan, and Geoffrey E. Hinton. Adaptive mixtures of local experts. *Neural Computation*, 3(1):79–87, 1991. doi: 10.1162/neco.1991.3.1.79.
- [23] Albert Q. Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, et al. Mixtral of experts. *arXiv preprint arXiv:2401.04088*, 2024.
- [24] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.
- [25] Jakub Krajewski, Jan Ludziejewski, Kamil Adamczewski, Maciej Piontkowski, Piotr Piotrowski, Szymon Antoniak, et al. Scaling laws for fine-grained mixture of experts. *arXiv preprint arXiv:2402.07871*, 2024.
- [26] Guokun Lai, Qizhe Xie, Hanxiao Liu, Yiming Yang, and Eduard Hovy. RACE: Large-scale reading comprehension dataset from examinations. In *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*, pages 785–794, 2017.
- [27] Zhenzhong Lan, Mingda Chen, Sebastian Goodman, Kevin Gimpel, Piyush Sharma, and Radu Soricut. ALBERT: A lite BERT for self-supervised learning of language representations. In *International Conference on Learning Representations*, 2020.
- [28] Dmitry Lepikhin, HyoukJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. GShard: Scaling giant models with conditional computation and automatic sharding. *arXiv preprint arXiv:2006.16668*, 2021.
- [29] Mike Lewis, Shruti Bhosale, Tim Dettmers, Naman Goyal, and Luke Zettlemoyer. BASE layers: Simplifying training of large, sparse models. In *International Conference on Machine Learning*, 2021.
- [30] Zeyu Liu, Tim Dettmers, Xi Lin, Veselin Stoyanov, and Xian Li. Towards a unified view of sparse feed-forward network in pretraining large language model. In Houda Bouamor, Juan Pino, and Kalika Bali, editors, *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 15038–15061, Singapore, December 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.emnlp-main.930. URL <https://aclanthology.org/2023.emnlp-main.930/>.
- [31] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101*, 2019.
- [32] Niklas Muennighoff, Luca Soldaini Yang, Dirk Groeneveld, Kyle Lo, Jacob Morrison, Peter Izsak, et al. OLMoE: Open mixture-of-experts language models. *arXiv preprint arXiv:2409.02060*, 2024.
- [33] Elizbar A Nadaraya. On estimating regression. *Theory of Probability & Its Applications*, 9(1): 141–142, 1964.
- [34] Vinod Nair and Geoffrey E. Hinton. Rectified linear units improve restricted boltzmann machines. In *Proceedings of the 27th International Conference on Machine Learning*, pages 807–814. Omnipress, 2010.

- [35] Huy Nguyen, Nhat Ho, and Alessandro Rinaldo. On least square estimation in softmax gating mixture of experts. *arXiv preprint arXiv:2402.02952*, 2024.
- [36] Denis Paperno, Germán Kruszewski, Angeliki Dufter, Quan Pham, Raffaella Bernardi, and Marco Baroni. The LAMBADA dataset: Word prediction requiring a broad discourse context. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics*, pages 1525–1534, 2016.
- [37] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9, 2019.
- [38] Jack W Rae, Sebastian Borgeaud, Trevor Cai, Katie Millican, Jordan Hoffmann, Francis Song, John Aslanides, Sarah Henderson, Roman Ring, Susannah Young, et al. Scaling language models: Methods, analysis & insights from training gopher. *arXiv preprint arXiv:2112.11446*, 2021.
- [39] Samyam Rajbhandari, Conglong Li, Zhewei Yao, Minjia Zhang, Reza Yazdani Aminabadi, Ammar Ahmad Awan, Jeff Rasley, and Yuxiong He. DeepSpeed-MoE: Advancing mixture-of-experts inference and training to power next-generation AI scale. *arXiv preprint arXiv:2201.05596*, 2022.
- [40] Stephen Roller, Sainbayar Sukhbaatar, Arthur Szlam, and Jason Weston. Hash layers for large sparse models. *Advances in Neural Information Processing Systems*, 34, 2021.
- [41] Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. Winogrande: An adversarial winograd schema challenge at scale. *Communications of the ACM*, 64(9):99–106, 2021.
- [42] Noam Shazeer. GLU variants improve transformer. *arXiv preprint arXiv:2002.05202*, 2020.
- [43] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarczyk, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *arXiv preprint arXiv:1701.06538*, 2017.
- [44] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019.
- [45] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-LM: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2020.
- [46] Luca Soldaini, Rodney Kinney, Akshita Bhagia, Dustin Schwenk, David Atkinson, Russell Authur, Ben Bogin, Khyathi Chandu, Jennifer Dumas, Yanai Elazar, Valentin Hofmann, Ananya Jha, Sachin Kumar, Li Lucy, Xinxin Lyu, Nathan Lambert, Ian Magnusson, Jacob Morrison, Niklas Muennighoff, Aakanksha Naik, Crystal Nam, Matthew Peters, Abhilasha Ravichander, Kyle Richardson, Zejiang Shen, Emma Strubell, Nishant Subramani, Oyvind Tafjord, Evan Walsh, Luke Zettlemoyer, Noah Smith, Hannaneh Hajishirzi, Iz Beltagy, Dirk Groeneveld, Jesse Dodge, and Kyle Lo. Dolma: an open corpus of three trillion tokens for language model pretraining research. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 15725–15788, Bangkok, Thailand, August 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.acl-long.840. URL <https://aclanthology.org/2024.acl-long.840/>.
- [47] Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. RoFormer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024.
- [48] Kimi Team, Yifan Bai, Yiping Bao, Guanduo Chen, Jiahao Chen, Ningxin Chen, Ruijue Chen, Yanru Chen, Yuankun Chen, Yutian Chen, et al. Kimi k2: Open agentic intelligence. *arXiv preprint arXiv:2507.20534*, 2025.

- [49] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. LLaMA: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.
- [50] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in Neural Information Processing Systems*, 30, 2017.
- [51] Lean Wang, Huazuo Gao, Chenggang Zhao, Xu Sun, and Damai Dai. Auxiliary-loss-free load balancing strategy for mixture-of-experts. *arXiv preprint arXiv:2408.15664*, 2024.
- [52] Geoffrey S Watson. Smooth regression analysis. *Sankhyā: The Indian Journal of Statistics, Series A*, pages 359–372, 1964.
- [53] Johannes Welbl, Nelson F. Liu, and Matt Gardner. Crowdsourcing multiple choice science questions. In Leon Derczynski, Wei Xu, Alan Ritter, and Tim Baldwin, editors, *Proceedings of the 3rd Workshop on Noisy User-generated Text*, pages 94–106, Copenhagen, Denmark, September 2017. Association for Computational Linguistics. doi: 10.18653/v1/W17-4413. URL <https://aclanthology.org/W17-4413/>.
- [54] Juntong Wu, Jialiang Cheng, Fuyu Lv, Ou Dan, and Li Yuan. Sere: Similarity-based expert re-routing for efficient batch decoding in moe models. *arXiv preprint arXiv:2602.07616*, 2026.
- [55] Mengzhou Xia, Tianyu Gao, Zhiyuan Zeng, and Danqi Chen. Sheared LLaMA: Accelerating language model pre-training via structured pruning. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=09i0dae0zp>.
- [56] Fuzhao Xue, Zian Zheng, Yao Fu, Jinjie Ni, Zangwei Zheng, Wangchunshu Zhou, and Yang You. OpenMoE: An early effort on open mixture-of-experts language models. *arXiv preprint arXiv:2402.01739*, 2024.
- [57] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. HellaSwag: Can a machine really finish your sentence? In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 4791–4800, 2019.
- [58] Biao Zhang and Rico Sennrich. Root mean square layer normalization. *Advances in Neural Information Processing Systems*, 32, 2019.
- [59] Peiyuan Zhang, Guangtao Zeng, Tianduo Wang, and Wei Lu. Tinyllama: An open-source small language model. *arXiv preprint arXiv:2401.02385*, 2024.
- [60] Susan Zhang, Stephen Roller, Naman Goyal, Mikel Artetxe, Moya Chen, Shuohui Chen, Christopher Dewan, Mona Diab, Xian Li, Xi Victoria Lin, et al. Opt: Open pre-trained transformer language models. *arXiv preprint arXiv:2205.01068*, 2022.
- [61] Zihao Zhao, Eric Wallace, Shi Feng, Dan Klein, and Sameer Singh. Calibrate before use: Improving few-shot performance of language models. In *International conference on machine learning*, pages 12697–12706. PMLR, 2021.
- [62] Chuanyang Zheng, Jiankai Sun, Yihang Gao, Enze Xie, Yuehao Wang, Peihao Wang, Ting Xu, Matthew Chang, Liliang Ren, Jingyao Li, et al. Understanding the mixture-of-experts with nadaraya-watson kernel. *arXiv preprint arXiv:2509.25913*, 2025.
- [63] Shu Zhong, Mingyu Xu, Tenglong Ao, and Guang Shi. Understanding transformer from the perspective of associative memory. *arXiv preprint arXiv:2505.19488*, 2025.
- [64] Yanqi Zhou, Tao Lei, Hanxiao Liu, Nan Du, Yanping Huang, Vincent Zhao, Andrew Dai, Zhifeng Chen, Quoc Le, and James Laudon. Mixture-of-experts with expert choice routing. *Advances in Neural Information Processing Systems*, 35, 2022.
- [65] Tong Zhu, Xiaoye Qu, Daize Dong, Jiacheng Ruan, Jingqi Tong, Conghui He, and Yu Cheng. LLaMA-MoE: Building mixture-of-experts from LLaMA with continual pre-training. In Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen, editors, *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 15913–15923, Miami, Florida, USA, November 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.emnlp-main.890. URL <https://aclanthology.org/2024.emnlp-main.890/>.

- [66] Barret Zoph, Irwan Bello, Sameer Kumar, Nan Du, Yanping Huang, Jeff Dean, Noam Shazeer, and William Fedus. ST-MoE: Designing stable and transferable sparse expert models. *arXiv preprint arXiv:2202.08906*, 2022.

A Limitations and Future Work

Scale of experiments. Our experiments are conducted at 182M–978M parameter scales with 30B training tokens. While the consistent improvement across five scales is encouraging, validating UNIPool at billion-parameter scales with longer training horizons is an important direction.

Throughput and memory. We do not report wall-clock throughput comparisons in this work. At the matched setting ($M = 8L$), UNIPool has the same total expert FFN count as vanilla MoE, so the architectural change is that all layers share a single pool by reference rather than that the parameter count itself decreases. Storage and memory savings emerge only in the reduced-pool regime (Section 5.2), where smaller pools achieve matched or better quality with strictly fewer expert parameters. The pool auxiliary loss also introduces a small overhead from cross-layer statistic accumulation, and routing into a larger candidate pool may affect token-dispatch efficiency under expert parallelism; a detailed throughput and expert-parallel scaling study is left for future work.

Downstream evaluation. We evaluate on seven zero-shot benchmarks (Section 5.3). A broader evaluation including few-shot settings would further strengthen the findings.

B Model and MoE Configurations

Table 6: Backbone configurations for the five evaluation scales. All models use dense-width FFNs (or expert FFNs for MoE variants) with intermediate size ($4 \times H$) and SwiGLU activation. “Active scale” denotes the dense-equivalent active parameter budget including embeddings; “Total Params” reports stored UNIPool parameters. MoE variants store additional expert parameters, with vanilla MoE and UNIPool matched in total expert budget.

Scale	Layers	Hidden	Heads	KV Heads	Seq Len	Active Scale	Total Params
182M	12	768	12	4	1024	~182M	777.9M
469M	24	1024	16	4	1024	~469M	2.588B
650M	36	1024	16	4	1024	~650M	3.834B
830M	48	1024	16	4	1024	~830M	5.081B
978M	24	1536	16	4	1024	~978M	5.742B

Table 7: MoE configuration comparison between vanilla MoE and UNIPool. The two variants are matched in total expert FFNs and per-token expert FLOPs.

	Vanilla MoE	UNIPool
Expert ownership	Per-layer	Global shared pool
Number of experts	8 per layer	$8 \times L$ (global pool)
Total expert FFNs	$8L$	$8L$
Expert evals per token	L	L
Routing	Top-1, softmax	Top-1, NormRouter
Per-layer aux loss	1×10^{-2}	0
Pool aux loss	—	$1\text{--}2 \times 10^{-2}$
Expert parallelism	1	1
Grouped GEMM	✓	✓

C Additional Training Curves

Figure 4 complements the endpoint validation losses in Section 5.2 by showing the full optimization trajectories. Across the 182M, 469M, and 650M scales, UNIPool stays below the matched vanilla MoE baseline after the initial warmup phase, indicating that the gain is not only a final-checkpoint artifact. At 182M, the gap opens early and widens steadily; at 469M, the two curves diverge

visibly after warmup and end with a validation-loss difference of roughly 0.035; at 650M, UNIPool continues to maintain a clear advantage throughout training.

Panel (d) reports the 182M sharing-scope ablation over training. The trajectory ordering mirrors the endpoint ablation results: global sharing ($G=1$) remains the lowest-loss configuration for most of training, vanilla MoE ($G=12$) is the highest-loss endpoint, and grouped sharing configurations ($G=2, 4, 6$) generally interpolate between them. This suggests that broader expert sharing improves the optimization trajectory itself, rather than merely selecting a better final checkpoint.

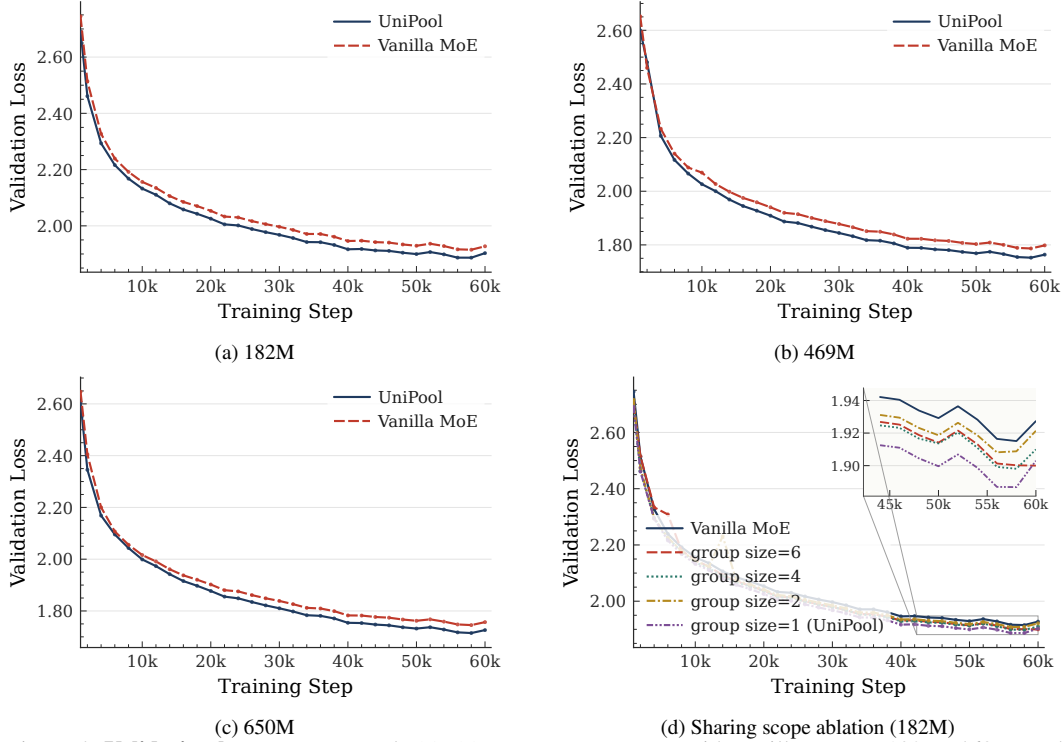


Figure 4: **Validation loss curves.** Panels (a)–(c) compare UNIPool with vanilla MoE at 182M, 469M, and 650M over 30B Pile tokens. Panel (d) shows the 182M sharing-scope ablation, where $G=1$ is full UNIPool and $G=12$ is vanilla MoE; grouped configurations interpolate between the endpoints.

D Hyperparameter Details

Table 8 provides complete hyperparameter details for all experimental configurations.

All models use RMSNorm [58], SwiGLU activation [42], rotary positional embeddings (RoPE) [47], grouped query attention with 4 KV heads, and untied input/output embeddings. Training uses Megatron-LM with sequence parallelism and distributed optimizer. Activation checkpointing with MoE layer recompute is enabled for the 469M, 650M, 830M, and 978M scales.

E Distinct-Expert Accounting

For a token x in an L -layer top-1 MoE model, let $e_l(x)$ denote the expert selected at layer l . In vanilla MoE, each layer owns a disjoint expert set. Thus, even if two layers choose the same local expert index, they access different parameter tensors, and the number of unique expert tensors touched by a token is exactly L .

In UNIPool, all layers route into a shared pool of M experts. The number of unique expert tensors touched by token x is

$$U(x) = |\{e_l(x) : l = 1, \dots, L\}|, \quad 1 \leq U(x) \leq L. \quad (8)$$

Table 8: Full hyperparameter details for all model scales.

Hyperparameter	182M	469M	650M	830M	978M
<i>Architecture</i>					
Number of layers	12	24	36	48	24
Hidden size	768	1024	1024	1024	1536
FFN intermediate size	3072	4096	4096	4096	6144
Attention heads	12	16	16	16	16
KV heads (GQA)	4	4	4	4	4
Sequence length	1024	1024	1024	1024	1024
Normalization	RMSNorm	RMSNorm	RMSNorm	RMSNorm	RMSNorm
Activation	SwiGLU	SwiGLU	SwiGLU	SwiGLU	SwiGLU
Position embedding	RoPE (base 1M)	RoPE (base 1M)	RoPE (base 1M)	RoPE (base 1M)	RoPE (base 1M)
Total parameters (UNIPOOL)	777.9M	2.588B	3.834B	5.081B	5.742B
<i>MoE (UNIPOOL)</i>					
Global expert pool size	96	192	288	384	192
Router top- k	1	1	1	1	1
Pool aux loss coeff	1×10^{-2}	2×10^{-2}	2×10^{-2}	2×10^{-2}	2×10^{-2}
Per-layer aux loss coeff	0	0	0	0	0
NormRouter	✓	✓	✓	✓	✓
Router init	Monte Carlo	Monte Carlo	Monte Carlo	Monte Carlo	Monte Carlo
<i>Training</i>					
Global batch size	512	512	512	512	512
Micro batch size	16	16	16	16	16
Training iterations	60,000	60,000	60,000	60,000	60,000
Total tokens	~30B	~30B	~30B	~30B	~30B
Learning rate	5×10^{-4}	5×10^{-4}	5×10^{-4}	5×10^{-4}	5×10^{-4}
Min learning rate	5×10^{-5}	5×10^{-5}	5×10^{-5}	5×10^{-5}	5×10^{-5}
LR schedule	Cosine	Cosine	Cosine	Cosine	Cosine
Warmup fraction	0.01	0.01	0.01	0.01	0.01
Gradient clipping	1.0	1.0	1.0	1.0	1.0
Precision	bf16	bf16	bf16	bf16	bf16
Init std	0.01	0.01	0.01	0.01	0.01

We report the validation-set average $\mathbb{E}_x[U(x)]$ and the normalized fraction $\mathbb{E}_x[U(x)]/L$ in Table 9. This metric summarizes how much cross-layer expert reuse emerges in the shared pool.

Table 9: Unique experts touched per token under top-1 routing; M is the shared-pool size.

Setting	L	M	Unique/ L
Full pool	12	96	11.29/12 (94.1%)
Reduced pool	12	64	11.46/12 (95.5%)
Reduced pool	12	48	11.31/12 (94.3%)
Full pool	24	192	21.48/24 (89.5%)
Reduced pool	24	96	20.79/24 (86.6%)
Full pool	36	288	30.03/36 (83.4%)
Reduced pool	36	128	30.12/36 (83.7%)

F Additional Routing-Randomization Details

Production MoE models: per-task results. Table 10 reports per-task downstream accuracy under the single-layer deep-half random-routing intervention for the three production MoE models discussed in Section 3. TOP-K denotes the model’s original learned top- k router and RANDOM denotes the mean accuracy after randomizing one deep-half MoE layer at a time and averaging across layers; Avg is the unweighted mean and drops are measured relative to TOP-K.

Matched randomization for shared experts. For vanilla MoE, the random-routing intervention samples uniformly from the 8 private experts owned by the selected layer. For UNIPOOL, uniform sampling over the full shared pool would not be comparable, because each layer can choose from $M = L \times 8$ experts rather than from 8 private experts. We therefore first identify each layer’s top-8 most-used shared experts on a held-out Pile validation split, then sample uniformly from that per-layer top-8 set during the intervention. This keeps the randomized choice set the same size as vanilla MoE

Table 10: Routing redundancy under single-layer randomization in production MoE models. Accuracy (%) is reported on five downstream benchmarks.

Model	Routing	ARC-E	ARC-C	PIQA	HellaSwag	WinoGrande	Avg
Qwen1.5-MoE	TOP-K	69.23	44.20	80.47	77.30	68.43	67.92
	RANDOM	66.76	42.19	79.07	76.08	67.34	66.29 (−1.6)
DeepSeek-V2-Lite	TOP-K	58.59	33.02	67.57	56.82	54.93	54.19
	RANDOM	57.23	32.08	65.88	55.41	54.57	53.03 (−1.2)
Qwen3-30B-A3B	TOP-K	79.50	55.97	80.79	77.70	71.11	73.02
	RANDOM	78.67	54.98	79.71	76.85	70.10	72.06 (−1.0)

while respecting the fact that different UNIPool layers can prefer different regions of the global pool. We also report the standard full-pool random protocol, where UNIPool samples uniformly from all shared experts.

Table 11: Routing-randomization results on our trained models. Accuracy (%) is reported on five downstream benchmarks. TOP-K: learned top- k routing; RANDOM: mean accuracy after randomizing one deep-half MoE layer at a time and averaging across layers. For UNIPool, $\dagger$ denotes the matched top-8 random protocol and unmarked RANDOM denotes the standard full-pool random protocol.

Model	Routing	ARC-E	ARC-C	PIQA	HellaSwag	WinoGrande	Avg
Vanilla MoE (469M)	TOP-K	44.70	25.09	65.94	38.63	51.14	45.10
	RANDOM	43.05	24.82	63.43	37.73	50.12	43.83 (−1.3)
UNIPool (469M)	TOP-K	47.39	25.94	69.10	40.64	52.72	47.16
	RANDOM	42.06	24.76	60.30	37.26	52.32	43.34 (−3.8)
	RANDOM †	41.61	25.21	60.62	37.46	50.61	43.10 (−4.1)
Vanilla MoE (978M)	TOP-K	48.65	26.45	68.88	44.24	52.41	48.13
	RANDOM	46.06	26.36	66.11	42.69	52.00	46.64 (−1.5)
UNIPool (978M)	TOP-K	49.03	25.60	70.24	44.73	52.17	48.35
	RANDOM	43.21	25.59	62.40	40.30	50.39	44.38 (−4.0)
	RANDOM †	42.33	25.29	62.13	40.40	51.10	44.25 (−4.1)

G Pool Auxiliary Loss: Detailed Derivation

Here we provide the full derivation showing that the pool-level loss decomposes into per-layer terms. Starting from the pool loss definition:

$$\mathcal{L}_{\text{pool}} = \alpha_{\text{pool}} \cdot M \cdot \sum_{i=1}^M \bar{f}_i \cdot \bar{P}_i \quad (9)$$

$$= \alpha_{\text{pool}} \cdot M \cdot \sum_{i=1}^M \bar{f}_i \cdot \frac{1}{L} \sum_{l=1}^L P_i^{(l)} \quad (10)$$

$$= \frac{1}{L} \sum_{l=1}^L \alpha_{\text{pool}} \cdot M \cdot \sum_{i=1}^M \bar{f}_i \cdot P_i^{(l)}. \quad (11)$$

The last step uses the fact that $\bar{f}_i$ does not depend on l . Each summand is the per-layer pool loss contribution, which can be computed independently.

One-step-behind computation. Computing $\bar{f}_i$ requires statistics from all layers, which are unavailable until the full forward pass completes. To avoid cross-layer tensor dependencies (which would break activation checkpointing), we use a *one-step-behind* scheme: each layer computes its pool loss contribution using $\bar{f}_i$ from the *previous* micro-batch. The global token distribution $\bar{f}_i$ is accumulated without gradients and updated after all layers complete their forward pass. Only the

routing probabilities $P_i^{(l)}$ carry gradients, so the pool loss only updates router parameters, not expert FFN parameters, through this path.

H NormRouter: Monte Carlo Initialization Details

The main text uses c as a fixed calibration factor for the NormRouter score scale. Given E experts and top- k routing, we choose c so that the initial selected scores have approximately unit magnitude:

$$c = \mathbb{E} \left[\frac{1}{\sqrt{\sum_{j=1}^k y_{(j)}^2}} \right], \quad y = \text{ReLU} \left(\frac{x}{\|x\|_2} \right), \quad x \sim \mathcal{N}(0, I_E), \quad (12)$$

where $y_{(j)}$ denotes the j -th largest component of y . Algorithm 1 estimates this expectation by Monte Carlo sampling at initialization time.

Algorithm 1: Monte Carlo estimation of NormRouter scale constant c

Input: Number of experts E , top- k hyperparameter k , number of samples $N = 10^5$

Output: Scale constant c

A set of samples $\mathcal{S} \leftarrow \emptyset$;

for $n = 1$ **to** N **do**

 Sample $\mathbf{x} \sim \mathcal{N}(0, \mathbf{I}_E)$;

$\mathbf{y} \leftarrow \text{ReLU}(\mathbf{x}/\|\mathbf{x}\|_2)$;

 Sort the elements in $\mathbf{y}$ in descending order to obtain $\tilde{\mathbf{y}}$, and take top- k components $\tilde{\mathbf{y}}_{:k}$;

 Append the element $1/\|\tilde{\mathbf{y}}_{:k}\|_2$ to $\mathcal{S}$;

end

$c \leftarrow \text{mean}(\mathcal{S})$;

return c
